Compatibilidade entre candidatas e vagas com Teoria dos Grafos: caminhos mínimos, ancestral comum e emparelhamento no ELAB

Caio Vitor[1], **David Rodrigues**[1], **Jonathan Alexandre**[1]
Igor Barbosa[1], **Isabelle Pascini**[1], **Mateus Batista**[1]

[1]Instituto Federal de Minas Gerais (IFMG) – Campus Sabará
Sabará – MG – Brasil

Abstract. *This paper presents the graph-theoretic core of ELAB, a platform that connects women in technology to job openings. Skills are modeled as a weighted undirected graph and areas as a tree. A multi-source Dijkstra search with edge cost* $-\ln w$ *measures skill similarity, a nearest-common-ancestor query measures area affinity, and the Hungarian method allocates candidates to openings through maximum-weight matching in a bipartite graph. On simulated data (100 candidates, 40 openings), the graph recognizes affinity in 68.5% of the 2,286 candidate–opening pairs that share no skill by name, and the ranking of openings stays stable under random perturbation of the weights. The weights are design choices and were not calibrated, and the experiments measure robustness, not accuracy.*

Resumo. *Este trabalho apresenta o núcleo de Teoria dos Grafos do ELAB, uma plataforma que conecta mulheres em tecnologia a vagas de emprego. As habilidades são modeladas como um grafo não direcionado e ponderado, e as áreas de atuação como uma árvore. Uma busca de Dijkstra multi-origem com custo de aresta* $-\ln w$ *mede a similaridade entre habilidades, o ancestral comum mais próximo mede a afinidade entre áreas, e o método húngaro distribui candidatas entre vagas por emparelhamento de peso máximo em um grafo bipartido. Em dados simulados (100 candidatas e 40 vagas), o grafo reconhece afinidade em 68,5% dos 2.286 pares candidata-vaga que não têm nenhuma habilidade em comum pelo nome, e o ranking de vagas se mantém estável sob perturbação aleatória dos pesos. Os pesos são escolhas de projeto, sem calibração, e os experimentos medem robustez, e não acerto.*

1. Introdução

O ELAB é uma rede profissional que conecta colaboradoras (mulheres em tecnologia) a empresas que publicam vagas. O projeto nasceu no curso de Sistemas de Informação do IFMG, campus Sabará, e a empresa fictícia dos dados de demonstração se chama Tech Sabará. A plataforma usa Teoria dos Grafos para responder a duas perguntas:

1. **Compatibilidade:** quanto o perfil de uma candidata combina com uma vaga? A vaga lista habilidades (cada uma com importância de 1 a 5 e a marca de obrigatória), área, nível, modalidade e faixa salarial.
2. **Alocação:** dadas várias candidatas e várias vagas, quem deve ficar com qual vaga para que o resultado total seja o melhor possível, com cada candidata em no máximo uma vaga e cada vaga com no máximo uma candidata?

Comparar habilidades pelo nome falha em casos óbvios: quem sabe TypeScript recebe nota zero numa vaga que pede JavaScript, embora as duas sejam muito próximas. Nos dados de demonstração, 57,1% dos pares candidata-vaga não têm nenhuma habilidade em comum (Seção 6), e a comparação por nome daria zero para todos eles. Um grafo de habilidades permite medir proximidade, e não só igualdade.

Intuitivamente, imagine um mapa em que cada habilidade é uma cidade e cada relação entre duas habilidades é uma estrada. Estradas fortes (JavaScript e TypeScript) são curtas e estradas fracas são longas. Para saber se alguém está perto de uma exigência da vaga, procuramos o caminho mais curto entre o que ela já sabe e o que a vaga pede.

O trabalho integra algoritmos clássicos (Dijkstra, ancestral comum em árvore e método húngaro) num problema de recomendação; não propõe algoritmo novo. Os dados são simulados e os pesos do modelo são escolhas de projeto, sem calibração. O contexto de Sabará aparece na motivação; o problema não depende de geografia, e não se modelou distância entre cidades. A Seção 2 apresenta o trabalho relacionado, a Seção 3 a modelagem, a Seção 4 os algoritmos, a Seção 5 a implementação e os dados, a Seção 6 os experimentos, a Seção 7 as limitações, a Seção 8 a ética e a Seção 9 conclui.

2. Trabalho relacionado

2.1. Grafos em redes sociais e recomendação

A análise de redes sociais como ferramenta para ligar relações entre indivíduos a fenômenos em larga escala, com a distinção entre laços fortes e fracos, é um marco clássico [Granovetter 1973]. No ELAB, a escala de pesos de 0,3 a 0,9 também expressa intensidade de relação, mas entre habilidades, e não entre pessoas. A previsão de ligações em redes sociais mede a proximidade entre dois nós apenas pela topologia do grafo [Liben-Nowell and Kleinberg 2007]; a similaridade entre habilidades do ELAB segue a mesma ideia, pois depende só da estrutura do grafo e dos pesos das arestas. Em sistemas reais, o serviço *Who To Follow* do Twitter recomenda a partir do grafo inteiro mantido na memória de um único servidor [Gupta et al. 2013], estratégia análoga à do cache em memória do ELAB, em escala muito menor. Para recomendação em escala de web, passeios aleatórios e convoluções em grafos aprendem a representação a partir dos dados [Ying et al. 2018]; aqui, ao contrário, os pesos são definidos manualmente.

2.2. Similaridade em hierarquias e ancestral comum

A similaridade conceitual entre dois conceitos de uma rede semântica hierárquica pode ser medida pela distância entre eles na hierarquia [Rada et al. 1989], noção usada na árvore de áreas (Seção 4.2). O decaimento $0{,}7^d$, que converte distância em afinidade, é uma escolha do grupo e não vem desse trabalho. Há medidas alternativas, baseadas no conteúdo de informação do ancestral comum [Resnik 1995] ou em sua profundidade [Wu and Palmer 1994]. Para achar o ancestral comum mais próximo existem algoritmos que respondem em tempo constante após um pré-processamento [Harel and Tarjan 1984, Bender and Farach-Colton 2000]; com 33 áreas e altura 2, uma subida simples pela árvore basta.

2.3. Caminhos mínimos e atribuição

O algoritmo de Dijkstra resolve caminhos mínimos com custos não negativos [Dijkstra 1959]. O desempenho depende da fila de prioridade: com heap de Fibo-

nacci chega-se a $O(E + V \log V)$ [Fredman and Tarjan 1987], mas o ELAB usa um heap binário, mais simples e suficiente para o tamanho do grafo. O método húngaro resolve o problema de atribuição [Kuhn 1955, Munkres 1957]. Técnicas de caminhos mínimos para transporte e fluxo de custo mínimo aparecem em [Tomizawa 1971, Edmonds and Karp 1972]; o resumo do primeiro informa um limite de n^3 operações para a atribuição $n \times n$.

2.4. Emparelhamento em mercados de trabalho e viés

O emparelhamento estável, em que nenhum par prefere mutuamente ficar junto a manter o que recebeu, foi definido por Gale e Shapley [Gale and Shapley 1962], e o mercado de médicos residentes é um estudo de caso de sua aplicação real [Roth 1984]. O ELAB adota outro critério: maximiza a soma das notas (emparelhamento de peso máximo), porque a plataforma tem uma nota única e comum, sem listas de preferência dos dois lados. A consequência é que o resultado não garante estabilidade. Sobre viés, há uma análise do que as empresas que vendem algoritmos de triagem de candidatos declaram sobre desenvolvimento, validação e mitigação de viés [Raghavan et al. 2019]; ela motiva as decisões da Seção 8.

3. Modelagem

O motor usa três estruturas, resumidas na Tabela 1.

Tabela 1. Estruturas do modelo e tamanho nos dados.

Estrutura	Vértices	Arestas	Tamanho
Grafo de habilidades	habilidades	similaridade em $(0, 1]$	142 vértices, 252 arestas
Árvore de áreas	áreas	pai-filho	33 vértices, altura 2
Grafo bipartido	candidatas e vagas	nota de 0 a 100	100×40

3.1. Grafo de habilidades

É um grafo não direcionado e ponderado, em que o peso de uma aresta é a similaridade entre as duas habilidades. Os dados têm 142 vértices e 252 arestas, grau médio de 3,5 e densidade de 2,5% (252 de 10.011 pares possíveis). Os maiores graus são os de Python e SQL (14), React (12), JavaScript e Node (10). Cada par é gravado uma única vez, sem laços, e a estrutura de dados simetriza as arestas ao carregar. Os pesos assumem só sete valores (Tabela 2), com média de 0,60.

Tabela 2. Distribuição dos pesos das 252 arestas.

Peso	0,3	0,4	0,5	0,6	0,7	0,8	0,9
Arestas	15	43	52	31	51	42	18

A escala adotada é: 0,9 para habilidades quase intercambiáveis (JavaScript e TypeScript); 0,7 para o mesmo ecossistema (JavaScript com React e com Node); 0,5 e 0,6 para alternativas da mesma categoria (JavaScript com Vue, 0,6; com Angular e Svelte, 0,5); e 0,3 e 0,4 para relações fracas.

Origem dos pesos. As relações e os pesos foram definidos a priori, por julgamento de plausibilidade; não foram estimados a partir de dados de uso (por exemplo, co-ocorrência de habilidades em currículos ou vagas) nem calibrados. É a premissa mais forte do modelo e é discutida na Seção 7.

Dois componentes conexos. O grafo tem 131 vértices num componente e 11 em outro (habilidades de gestão e comunicação, como Scrum, liderança, inglês e Jira). Não há aresta entre os dois, então a afinidade entre uma habilidade de gestão e uma habilidade técnica é sempre zero.

3.2. Árvore de áreas

A hierarquia de áreas de atuação tem 33 nós: uma raiz (Tecnologia da Informação), 13 filhas (por exemplo, Backend, Frontend e Dados) e 19 netas (por exemplo, Backend Node.js e Frontend React), portanto altura 2.

3.3. Grafo bipartido de alocação

De um lado ficam as candidatas, do outro as vagas, e cada aresta tem como peso a nota de compatibilidade (inteiro de 0 a 100). Um emparelhamento é um conjunto de arestas em que nenhum vértice aparece duas vezes.

4. Algoritmos

4.1. Similaridade entre habilidades: Dijkstra

Intuitivamente, é o GPS do mapa de habilidades: colocamos a candidata em todas as habilidades que ela já tem e andamos primeiro pelas estradas mais fortes.

O algoritmo de Dijkstra soma custos e busca o menor total, mas queremos o caminho de maior similaridade, que se obtém multiplicando pesos. A ponte entre as duas coisas é o custo de aresta

$$c(w) = -\ln w, \qquad w \in (0, 1]. \tag{1}$$

Seja um caminho com arestas de pesos $w_1, \ldots, w_k \in (0, 1]$. Sua similaridade é $\prod_i w_i$. Como o logaritmo é estritamente crescente, maximizar o produto equivale a minimizar $\sum_i -\ln w_i$. Como cada $w_i \leq 1$, cada custo $c(w_i)$ é maior ou igual a zero, que é a condição exigida por Dijkstra. Logo, o caminho de menor custo é o de maior produto de pesos, e a similaridade entre duas habilidades é

$$\mathrm{sim}(u, v) = e^{-d(u,v)}, \tag{2}$$

em que d é a distância de menor custo. Por exemplo, de TypeScript a React passando por JavaScript (pesos 0,9 e 0,7), os custos são $0{,}105 + 0{,}357 = 0{,}462$ e a similaridade é $e^{-0{,}462} = 0{,}63 = 0{,}9 \times 0{,}7$.

Todas as habilidades que a candidata já tem entram como origens com distância 0, então uma única execução por candidata serve para todas as vagas. Habilidade que ela tem vale 1; habilidade fora do alcance vale 0. O corte de similaridade mínima 0,25 equivale a um custo máximo de $-\ln 0{,}25 \approx 1{,}386$, e vértices além dele são descartados. Como o menor peso do grafo é 0,3, toda aresta direta passa no corte; um caminho de duas arestas só passa se o produto for pelo menos 0,25 (0,5 e 0,5 passa; 0,3 e 0,8 não).

A complexidade é $O((V+E)\log V)$ por execução, com um heap binário próprio. A implementação não tem a operação de diminuir chave: insere entradas repetidas e descarta as obsoletas ao retirá-las. Aqui $V = 142$ e $E = 252$. Uma consequência do modelo é que só a habilidade mais próxima conta; várias habilidades vizinhas não somam evidências.

4.2. Afinidade entre áreas: ancestral comum mais próximo

Intuitivamente, é um organograma: para saber quão perto estão duas áreas, contamos quantos passos é preciso subir até um ancestral comum e descer de novo.

Para cada área monta-se a lista de ancestrais até a raiz. O primeiro ancestral da área A que também aparece na lista de B é o ancestral comum mais próximo, e a distância d é a posição dele na lista de A somada à posição na lista de B. A afinidade é $0{,}7^d$, que vale 1 para a mesma área e 0 se não houver ancestral comum (Tabela 3). É uma subida simples, de custo $O(h)$ por consulta, com h a altura da árvore (aqui, 2). Por exemplo, Backend Node.js e Backend têm distância 1; Backend Node.js e Backend Java, 2; Backend Node.js e Frontend React, 4, a maior distância possível nesta árvore.

Tabela 3. Afinidade de área em função da distância na árvore.

Distância d	0	1	2	3	4
Afinidade $0{,}7^d$	1,00	0,70	0,49	0,34	0,24

4.3. Nota de compatibilidade

A nota combina quatro componentes entre 0 e 1: habilidades (H), área (A), nível (N) e preferências (P):

$$S = 100 \cdot \lambda \cdot (0{,}55\,H + 0{,}20\,A + 0{,}15\,N + 0{,}10\,P), \tag{3}$$

em que $\lambda = 0{,}7$ se algum requisito obrigatório tiver similaridade menor que 0,5, e $\lambda = 1$ caso contrário. Os pesos somam 1 e a nota é arredondada para inteiro. A Tabela 4 resume cada componente.

Tabela 4. Componentes da nota de compatibilidade.

Componente	Peso	Cálculo
Habilidades	55%	Média das similaridades dos requisitos, ponderada pela importância (padrão 3). Vaga sem requisitos: 0,5.
Área	20%	Vaga sem área: 0,5. Candidata sem área: 0,4. Senão, o máximo, entre as áreas da candidata, de afinidade $\times\,(0{,}6 + 0{,}4 \cdot$ interesse$/5)$.
Nível	15%	Escala estágio, júnior, pleno, sênior, especialista. Igual: 1. Candidata acima da vaga: $\max(0{,}5;\ 1 - 0{,}15\,d)$. Abaixo: $\max(0;\ 1 - 0{,}4\,d)$, com d a diferença de níveis.
Preferências	10%	Média entre modalidade (incompatível vale 0, ou 0,5 se um dos lados é híbrido) e salário $(\min(1,$ salário máximo$/$pretensão$))$.

Os pesos 55/20/15/10, o fator 0,7, o corte 0,25 e as fórmulas de nível e preferências são escolhas de projeto, sem calibração. O requisito chamado eliminatório no código só aciona o fator λ; não exclui ninguém.

4.4. Trilha de aprendizado

Intuitivamente, é um GPS de estudo: para cada habilidade que a vaga pede e a candidata ainda não domina, mostra o caminho a partir de uma habilidade que ela já tem, passando por habilidades intermediárias. A trilha reaproveita o resultado do Dijkstra: para cada requisito com similaridade menor que 1, reconstrói o caminho pelos predecessores. Os itens são ordenados por importância e, em empate, por similaridade, e a lista devolve no máximo 5. Um alvo fora do corte de 0,25 aparece como não alcançável.

4.5. Alocação ótima: método húngaro

Escolher sempre a melhor nota disponível pode estragar o conjunto. Com as notas 9080 8520, pegar primeiro o 90 obriga a segunda candidata a ficar com o 20, total 110; trocar as duas (80 + 85) dá 165. O método húngaro encontra o melhor total [Kuhn 1955, Munkres 1957].

O problema é um emparelhamento de peso máximo no grafo bipartido de candidatas e vagas. O método resolve a atribuição minimizando custos; o ELAB maximiza as notas passando seus negativos. A versão implementada usa potenciais (variáveis duais) e caminhos aumentantes de custo mínimo, da família de técnicas de [Tomizawa 1971, Edmonds and Karp 1972], com custo $O(n^2m)$ para n linhas e m colunas, $n \leq m$. Para matriz retangular, transpõe quando há mais linhas que colunas, e o custo passa a ser $O(\min^2 \times \max)$; com 100 candidatas e 40 vagas são cerca de 160 mil operações elementares. O parâmetro de nota mínima (40 na alocação) descarta, *depois* de otimizar, os pares com nota abaixo dele (ver Seção 7).

4.6. Cache do grafo

O grafo, as áreas e o motor são montados uma vez a partir do banco e mantidos em memória por processo, de modo que requisições simultâneas não carregam o grafo duas vezes. Uma função de invalidação força a recarga depois de alterar habilidades, arestas ou áreas, e uma habilidade ou aresta nova entra no cálculo sem mudar código.

5. Implementação, dados e reprodução

O backend é em Node.js, com MySQL, e o código dos grafos fica em um diretório próprio (Tabela 5).

Tabela 5. Conceito e arquivo correspondente.

Conceito	Arquivo
Grafo não direcionado e ponderado	`graph/Grafo.js`
Fila de prioridade (heap binário)	`graph/MinHeap.js`
Dijkstra multi-origem com poda	`graph/dijkstra.js`
Árvore e ancestral comum	`graph/arvoreAreas.js`
Método húngaro	`graph/hungaro.js`
Nota, trilha, ranking e alocação	`graph/motorMatch.js`
Cache do grafo	`services/grafo.service.js`

Os testes automatizados fazem 13 verificações em grafos pequenos de resposta conhecida: heap, simetria do grafo, Dijkstra (maior produto de pesos, multi-origem e

poda), ancestral comum e afinidade, método húngaro (igual à força bruta em 200 matrizes aleatórias retangulares) e o motor de nota. Eles mostram que as implementações acertam nesses casos; não provam a utilidade das notas.

Dados. O catálogo (33 áreas, 142 habilidades e 252 relações com peso) fica em três arquivos JSON elaborados para o projeto. Os dados de demonstração são **simulados**, gerados com semente fixa: 8 empresas, 40 vagas, 100 colaboradoras e 429 candidaturas. Não há pessoas reais, e a mesma semente produz sempre o mesmo resultado. O experimento usa 100 candidatas com habilidades, 40 vagas ativas com requisitos e 4.000 pares.

Reprodução. Com MySQL ou MariaDB rodando, um banco chamado `elab` e Node.js instalado, no diretório do backend:

```
npm install
cp .env.example .env
npm run migrate
npm run seed
npm run seed:demo
node scripts/experimento-baseline.js
node scripts/experimento-sensibilidade.js
```

Os dois scripts de experimento só leem o banco e não precisam do servidor web. O pacote de entrega traz as instruções completas.

6. Experimentos

Não há gabarito de relevância (registro de quem deveria ficar com cada vaga), então os experimentos medem efeitos observáveis: a cobertura (quanta afinidade o grafo enxerga onde a comparação por nome vê zero) e a robustez (como a nota reage quando uma habilidade some do perfil ou quando os pesos mudam). A comparação é feita só no componente de habilidades, que é onde o grafo atua, contra dois baselines sem grafo: a *cobertura ponderada* (soma das importâncias dos requisitos que a candidata tem, dividida pela soma total) e o índice de Jaccard [Jaccard 1901] entre o conjunto de habilidades da candidata e o de requisitos da vaga, que ignora importâncias.

6.1. Resgate e divergência de ranking

Dos 4.000 pares, 2.286 (57,1%) não têm nenhuma habilidade em comum, e nesses a cobertura e o Jaccard dão zero. Nesses mesmos pares, o grafo reconhece alguma afinidade em 1.566 (68,5%), com nota média de habilidades de 19,7 em 100. Exemplos de afinidades que os baselines não enxergam: TypeScript e JavaScript, MySQL e SQL, SQL e PostgreSQL (todas com 0,90). A sobreposição média do top-5 de vagas por candidata é de 76,6% entre grafo e cobertura e de 71,0% entre grafo e Jaccard, ou seja, o grafo muda entre 23% e 29% das cinco primeiras vagas.

6.2. Habilidade oculta

Para cada par em que a candidata tem ao menos um requisito da vaga, esconde-se do perfil a habilidade de maior importância que ela tem e a vaga pede, e mede-se o que acontece com aquela vaga no ranking das 40 vagas. São 1.714 casos (Tabela 6).

Tabela 6. Habilidade oculta (1.714 casos).

Método	Piora média no ranking	Nota retida	Continua no top-10
Grafo	4,29	66,6%	67,5%
Cobertura	8,81	25,8%	49,8%
Jaccard	8,43	28,8%	55,8%

6.3. Sensibilidade aos pesos

Se mexermos um pouco nos números do modelo, o ranking de vagas muda muito ou pouco? Cada configuração faz 30 tentativas com sementes fixas, para as 100 candidatas e as 40 vagas. Nas perturbações das arestas, cada aresta ganha ou perde um valor fixo, sorteado aresta a aresta, com o peso limitado a $[0{,}1;\ 1]$; como os pesos originais variam em passos de 0,1, a perturbação de 0,10 desloca cada relação para um nível vizinho da escala. Na perturbação dos pesos do score, cada peso é multiplicado por um fator sorteado entre 0,8 e 1,2. A métrica é a sobreposição do top-5 de vagas de cada candidata com o top-5 original; ao acaso, a sobreposição esperada entre 40 vagas é de 12,5%.

Tabela 7. Estabilidade do ranking sob perturbação dos pesos (30 tentativas).

Perturbação	Top-5 médio (dp)	Pior tentativa	Mesma vaga em 1º
Arestas $\pm 0{,}05$	94,8% (1,1)	92,6%	96,9%
Arestas $\pm 0{,}10$	90,5% (1,4)	88,0%	94,5%
Arestas $\pm 0{,}20$	84,9% (1,8)	80,2%	92,4%
Pesos do score $\pm 20\%$	97,9% (0,7)	96,4%	96,5%

Mesmo deslocando todas as arestas em até dois níveis da escala, o top-5 médio mantém 84,9% de sobreposição, e a pior tentativa fica em 80,2%, muito acima do acaso. Variar os pesos do score em 20% muda ainda menos o ranking.

6.4. O que os experimentos mostram

Mostram que o grafo enxerga afinidades que a comparação por nome ignora, que sua nota é mais estável quando uma habilidade falta e que o ranking não depende do valor exato de cada peso dentro das faixas testadas. **Não mostram** que o grafo acerta mais, por três motivos: não há gabarito de relevância; o teste de habilidade oculta favorece o grafo por construção (ao esconder uma habilidade, as vizinhas continuam valendo, então a nota cair menos prova robustez, e não acerto); e os dados são sintéticos.

7. Ameaças à validade e limitações

Construto. A nota de compatibilidade não foi validada contra nenhum resultado real (contratação, desempenho, satisfação).

Interna. As arestas e os pesos foram definidos a priori. A sensibilidade a perturbações aleatórias foi testada, mas só mede a estabilidade do ranking em relação a ele mesmo, e não a correção das relações; as perturbações são simétricas e não cobrem erro sistemático nem mudança de estrutura, e o corte 0,25, o fator 0,7 e o decaimento $0{,}7^d$ não foram variados.

Externa. Os dados são simulados por uma semente fixa, com 100 candidatas e 40 vagas; a distribuição de habilidades não reflete o mercado real, e os percentuais da Seção 6 dependem dela.

Conclusão. A execução é determinística e não há intervalos de confiança nem testes estatísticos. Só o componente de habilidades foi comparado.

Limitações do modelo.

- Os pesos 55/20/15/10, o fator 0,7, o corte 0,25 e o decaimento $0{,}7^d$ são escolhas de projeto, sem calibração.
- As 11 habilidades de gestão e comunicação nunca têm afinidade com as técnicas, pois formam um componente isolado.
- Só a habilidade mais próxima conta; os anos de experiência não entram no cálculo.
- A nota mínima da alocação é aplicada depois da otimização, então o resultado não é o melhor possível sujeito ao mínimo. Com a matriz $\begin{smallmatrix}70 & 60\\ 45 & 39\end{smallmatrix}$ e mínimo 40, o método húngaro escolhe $70 + 39 = 109$ e o par de 39 é descartado, sobrando 70; a alternativa $60 + 45 = 105$ teria os dois pares válidos.
- As notas são arredondadas para inteiros antes da alocação, o que cria empates, e o emparelhamento de peso máximo não garante estabilidade.
- O cache é por processo e a invalidação é manual.

Trabalhos futuros. Estimar os pesos a partir de dados reais (co-ocorrência de habilidades em currículos e vagas) e validar contra um gabarito; ligar os dois componentes do grafo de habilidades; corrigir o filtro de nota mínima zerando as notas abaixo do mínimo antes de otimizar; ampliar a sensibilidade para o corte, o fator 0,7, o decaimento e a estrutura do grafo; e comparar a alocação de peso máximo com o emparelhamento estável [Gale and Shapley 1962].

8. Ética e transparência

A nota é apoio, e não filtro: a plataforma mostra, para cada vaga, de onde vem a nota (habilidades, área, nível e preferências) e a situação de cada requisito, e o fator 0,7 do requisito obrigatório reduz a nota, mas não elimina a candidata. A recomendação de uso é não decidir por corte automático de nota. Cada componente e o caminho no grafo que justifica uma afinidade são devolvidos pela API, o que permite conferir o cálculo; a literatura sobre triagem algorítmica aponta a falta de transparência e de validação como problemas centrais [Raghavan et al. 2019].

O componente de preferências compara a pretensão salarial com o salário máximo da vaga. Pretensões salariais podem refletir desigualdades históricas de remuneração, algo especialmente relevante numa plataforma voltada a mulheres; por isso o peso desse componente é pequeno (10%) e deve ser revisto antes de uso real. O modelo não usa gênero nem outros atributos protegidos no cálculo da nota, e todos os dados dos experimentos são simulados.

9. Conclusão

O trabalho modelou habilidades como um grafo ponderado, áreas como uma árvore e a alocação como um emparelhamento bipartido, e resolveu cada parte com um algoritmo clássico cuja complexidade e cuja implementação foram descritas. A transformação $-\ln w$ permite usar Dijkstra para maximizar um produto de similaridades. Em dados

simulados, o grafo reconhece afinidades que a comparação por nome ignora e produz rankings estáveis sob perturbação dos pesos. A principal limitação é que os pesos foram definidos a priori e as medidas disponíveis não comprovam acerto; o próximo passo natural é estimar os pesos a partir de dados reais e validar a nota contra um gabarito.

Referências

Bender, M. A. and Farach-Colton, M. (2000). The LCA problem revisited. In *LATIN 2000: Theoretical Informatics*, volume 1776 of *Lecture Notes in Computer Science*, pages 88–94. Springer.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271.

Edmonds, J. and Karp, R. M. (1972). Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264.

Fredman, M. L. and Tarjan, R. E. (1987). Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3):596–615.

Gale, D. and Shapley, L. S. (1962). College admissions and the stability of marriage. *The American Mathematical Monthly*, 69(1):9–15.

Granovetter, M. S. (1973). The strength of weak ties. *American Journal of Sociology*, 78(6):1360–1380.

Gupta, P., Goel, A., Lin, J., Sharma, A., Wang, D., and Zadeh, R. (2013). WTF: The who to follow service at Twitter. In *Proceedings of the 22nd International Conference on World Wide Web (WWW)*, pages 505–514.

Harel, D. and Tarjan, R. E. (1984). Fast algorithms for finding nearest common ancestors. *SIAM Journal on Computing*, 13(2):338–355.

Jaccard, P. (1901). Étude comparative de la distribution florale dans une portion des Alpes et des Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37(142):547–579.

Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97.

Liben-Nowell, D. and Kleinberg, J. (2007). The link-prediction problem for social networks. *Journal of the American Society for Information Science and Technology*, 58(7):1019–1031.

Munkres, J. (1957). Algorithms for the assignment and transportation problems. *Journal of the Society for Industrial and Applied Mathematics*, 5(1):32–38.

Rada, R., Mili, H., Bicknell, E., and Blettner, M. (1989). Development and application of a metric on semantic nets. *IEEE Transactions on Systems, Man, and Cybernetics*, 19(1):17–30.

Raghavan, M., Barocas, S., Kleinberg, J., and Levy, K. (2019). Mitigating bias in algorithmic hiring: Evaluating claims and practices. arXiv preprint arXiv:1906.09208.

Resnik, P. (1995). Using information content to evaluate semantic similarity in a taxonomy. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 448–453.

Roth, A. E. (1984). The evolution of the labor market for medical interns and residents: A case study in game theory. *Journal of Political Economy*, 92(6):991–1016.

Tomizawa, N. (1971). On some techniques useful for solution of transportation network problems. *Networks*, 1(2):173–194.

Wu, Z. and Palmer, M. (1994). Verb semantics and lexical selection. In *Proceedings of the 32nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 133–138.

Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W. L., and Leskovec, J. (2018). Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 974–983.